

SIGNAL PROCESSING IN C++

Using Aquila 3.0

JAI SACHITH PAUL

When it comes to modern information engineering, communication and signal processing are birds of a feather. Whether it is broadcast and communication, audio and video recording, RADAR engineering, remote sensing, biomedical engineering or seismic data processing, a communication engineer has to deal with many signals.

In this issue, we introduce you to Aquila, an open source, cross-platform C++ library that helps the user perform all major digital signal processing (DSP) functions including finding various transformations, filtering operations, spectrogram computations and much more. The library, actively developed by Zbigniew Siciarz, is in its third stage of development.

What Aquila is capable of

If you would like to calculate the spectra of a signal using Fast Fourier Transforms (FFTs), compare two signals or extract features from a speech signal, this DSP library could be the appropriate solution for you.

Accepting the input. Aquila allows the user to input the signal in various data formats. It could be in the form of a text file or a .wav file. It supports the input in the form of an array or a vector in C++. The user can even input data in the form of a raw pulse code modulated data.

Signal and noise generators. The signals commonly used in DSP like sine wave, rectangular or square wave, triangular wave or saw tooth could easily be generated.

The library also contains com-

```
#include "aquila/global.h"
#include "aquila/source/generator/SineGenerator.h"
#include "aquila/transform/FftFactory.h"
#include "aquila/tools/TextPlot.h"
#include <algorithm>
#include <functional>
#include <memory>

int main()
{
    // input signal parameters
    const std::size_t SIZE = 64;
    const Aquila::FrequencyType sampleFreq = 2000;
    const Aquila::FrequencyType f1 = 96, f2 = 813;
    const Aquila::FrequencyType f_lp = 500;

    Aquila::SineGenerator sineGenerator1(sampleFreq);
    sineGenerator1.setAmplitude(32).setFrequency(f1).generate(SIZE);
    Aquila::SineGenerator sineGenerator2(sampleFreq);
    sineGenerator2.setAmplitude(8).setFrequency(f2).setPhase(0.75).generate(SIZE);
    auto sum = sineGenerator1 + sineGenerator2;

    Aquila::TextPlot plt("Signal waveform before filtration");
    plt.plot(sum);
}
```

Fig. 1: A sample code using Aquila

The story behind Aquila

The project was initially started in 2007 as an academic computer project, capable of recognising the songs of birds. In due course of development, the software was thoroughly redesigned and expanded in order to incorporate more general DSP tools. In Aquila 2, complexity of the code was greatly reduced by separating the code for signal processing from the graphical interface. The latest version of the software, Aquila 3.0, is intended to be more generic in handling data.

mon noise generators that include a white-noise generator and a pink-noise generator.

White noise is a random signal with constant power spectral density, whereas for pink noise the power spectral density is inversely proportional to the signal frequency.

Windowing functions. Windowing functions find applications in spectral analysis and beam-forming filter designing. When we deal with the windows in signal processing, these are zero valued outside chosen intervals. Various window functions such as rectangular window, Ham-

ming and Hanning windows, Barlett and Blackman could be easily implemented using this library.

Fast Fourier Transform. FFT has made signal processing in frequency domain as computationally feasible as in temporal or spatial domain. The user can find both the forward and the inverse FFT using the tool. He or she can have a visualisation of the frequency spectrum of sound or any other signal, with respect to time or other variables with the help of a spectrogram.

Filters. Application of filters on a signal essentially changes the shape